

A Sentinel for Null-Terminated Strings

Document #: P3705R2
Date: 2025-06-18
Project: Programming Language C++
Audience: SG-9 Ranges
LEWG
Reply-to: Eddie Nolan
[<eddiej Nolan@gmail.com>](mailto:eddiej Nolan@gmail.com)

Contents

- 1 Motivation
- 2 Naming discussion
 - 2.1 Bad names
 - 2.1.1 `c_str`
 - 2.1.2 `zstring_sentinel` and `zstring`
 - 2.2 Good names
 - 2.2.1 Status quo: `null_sentinel` and `null_term`
 - 2.2.2 `zero_termination_sentinel`
 - 2.2.3 Your suggestion here
- 3 Wording
 - 3.1 Exposition-only helper concept *default-initializable-and-equality-comparable-iter-value*
 - 3.2 Null sentinel
 - 3.2.1 Operations
 - 3.3 `null_term` CPO
 - 3.4 Feature test macro
- 4 History
 - 4.1 Changelog
 - 4.1.1 Changes since P2728R1
 - 4.1.2 Changes since P2728R3
 - 4.1.3 Changes since P2728R4

- 4.1.4 Changes since P2728R5
- 4.1.5 Changes since P2728R6
- 4.1.6 Changes since P2728R7
- 4.1.7 Changes since P3705R0
- 4.1.8 Changes since P3705R1
- 4.2 Relevant Polls/Minutes
 - 4.2.1 SG9 review of P3705R1 on 2025-06-18 during Sofia 2025
 - 4.2.2 SG9 review of D2728R4 on 2023-06-12 during Varna 2023
 - 4.2.3 SG16 review of P2728R3 on 2023-05-10 (Telecon)
- 5 References

§ 1 Motivation

Consider using `std::string::take` to get the first five characters from a string. If we pass a string literal with five or more characters, it looks okay:

```
static_assert(std::string{std::from_range, "Brubeck" |
std::views::take(5)} == "Brube"); // passes
```

However, if we pass it a string literal with fewer than five characters, we get in trouble:

```
// static_assert(std::string{std::from_range, "Dave" |
std::views::take(5)} == "Dave"); // fails
using namespace std::string_view_literals;
static_assert(std::string{std::from_range, "Dave" | std::views::take(5)}
== "Dave\0"sv); // passes
```

The reason the null terminator is included in the output here is because undecayed string literals are arrays of characters, including the null terminator, which the ranges library treats like any other array:

```
#include <algorithm>
#include <array>
#include <type_traits>

static_assert(std::is_same_v<std::remove_reference_t<decltype("foo")>,
const char[4]>);
static_assert(std::ranges::equal("foo", std::array{'f', 'o', 'o', '\0'}));
```

A common workaround is to wrap the string literal in `std::string_view`. But this has a performance cost: despite the fact that we only care about the first five characters, we still need to make a pass through the entire string to find the null terminator:

```
constexpr std::string take_five(char const* long_string) {
std::string_view const long_string_view = long_string; // read all of
long_string!
```

```

        return std::string{std::from_range, long_string_view |
        std::views::take(5)};
}

```

This paper introduces `null_sentinel` to solve this problem. It ends the range when it encounters a null:

```

constexpr std::string take_five(char const* long_string) {
    std::ranges::subrange const long_string_range(long_string,
    std::null_sentinel); // lazy!
    return std::string{std::from_range, long_string_range |
    std::views::take(5)};
}

```

It also introduces a `null_term` CPO for the common case of constructing a subrange like the one in the above example:

```

constexpr std::string take_five(char const* long_string) {
    return std::string{std::from_range, std::views::null_term(long_string) |
    std::views::take(5)};
}

```

The sentinel type matches any iterator position it at which `*it` is equal to a value-initialized object of type `iter_value_t<I>`. This works for null-terminated strings, but can also serve as the sentinel for any range terminated by a value-initialized value.

For example, `null_term` can be used to iterate `argv` and `environ`. The following program demonstrates this:

```

#include <print>

extern char** environ;

int main(int argc, char** argv) {
    std::println("argv: {}", std::views::null_term(argv));
    std::println("environ: {}", std::views::null_term(environ));
}

```

Output:

```

$ env --ignore-environment F00=bar BAZ=quux ./test corge
argv: ["/test", "corge"]
environ: ["F00=bar", "BAZ=quux"]

```

§ 2 Naming discussion

§ 2.1 Bad names

§ 2.1.1 `c_str`

`range-v3` provides similar functionality to `null_term` using the name `c_str`— unlike `null_term`, `c_str` only works with `char`, `wchar_t`, and so forth.

The name `c_str` would not make sense here because, for example, `std::views::c_str(argv)` is nonsensical and misleading as to what the range is doing (terminating the sequence of strings rather than terminating the strings themselves).

§ 2.1.2 `zstring_sentinel` and `zstring`

An example from Barry Revzin’s [P2210R2] provides similar functionality under the names `zstring_sentinel` and `zstring`.

I dislike this name for similar reasons as `c_str(argv)` is not a “string.”

Additionally, it invites confusion with [P3655R1?] `zstring_view`.

§ 2.2 Good names

§ 2.2.1 Status quo: `null_sentinel` and `null_term`

The advantages of these names are that they are terse, they do not include the word “string,” and the value-initialized `iter_value_t` value is referred to as “null,” which reflects the language we use in the standard (“null terminated byte string,” “null terminated multibyte string,” “null terminated character sequence,” “null pointer”)

§ 2.2.2 `zero_termination_sentinel`

This is used by think-cell’s [implementation](#) of this utility. I like it slightly less because it’s more verbose but also consider it acceptable.

§ 2.2.3 Your suggestion here

This is not an exhaustive list.

§ 3 Wording

§ 3.1 Exposition-only helper concept *default-initializable-and-equality-comparable-iter-value*

```
namespace std {  
  
    template<class I>  
    concept default-initializable-and-equality-comparable-iter-value =  
        default_initializable<iter_value_t<I>> &&  
            equality_comparable_with<iter_reference_t<I>, iter_value_t<I>>; //  
        exposition only  
  
}
```

§ 3.2 Null sentinel

```
namespace std {

    struct null_sentinel_t {
        template<input_iterator I>
            requires default-initializable-and-equality-comparable-iter-value<I>
        friend constexpr bool operator==(I const& it, null_sentinel_t);
    };

    inline constexpr null_sentinel_t null_sentinel;

}
```

§ 3.2.1 Operations

```
template<input_iterator I>
    requires default-initializable-and-equality-comparable-iter-value<I>
friend constexpr bool operator==(I const& it, null_sentinel_t);
```

Effects:

Equivalent to `return *it == iter_value_t<I>();`.

§ 3.3 null_term CPO

```
namespace std::views {

    inline constexpr unspecified null_term;

}
```

The name `null_term` denotes a customization point object ([customization.point.object]). Given a subexpression `E`, the expression `null_term(E)` is expression-equivalent to `ranges::subrange(E, null_sentinel)`.

§ 3.4 Feature test macro

Header `<version>` synopsis [version.syn]

```
#define __cpp_lib_ranges_null_sentinel 2026XXL
```

§ 4 History

§ 4.1 Changelog

This proposal was originally written by Zach Laine as part of [P2728R0], then updated and split out by Eddie Nolan.

§ 4.1.1 Changes since P2728R1

- Generalize `null_sentinel_t` to a non-Unicode-specific facility.

§ 4.1.2 Changes since P2728R3

- Change `null_sentinel_t` back to being Unicode-specific.

§ 4.1.3 Changes since P2728R4

- Move `null_sentinel_t` to `std`, remove its base member function, and make it useful for more than just pointers, based on SG-9 guidance.

§ 4.1.4 Changes since P2728R5

- Simplify the complicated constraint on the comparison operator for `null_sentinel_t`.

§ 4.1.5 Changes since P2728R6

- Fix a bug in `null_sentinel_t` causing it not to satisfy `sentinel_for` by changing its `operator==` to return `bool`.
- Fix a bug in `null_sentinel_t` where it did not support non-copyable input iterators by having `operator==` take input iterators by reference.

§ 4.1.6 Changes since P2728R7

- Move `null_sentinel` and `null_term` into P3705

§ 4.1.7 Changes since P3705R0

- Fix off-by-one in textual description in motivation section
- Add example with `argv` and `environ`
- Add `unchecked_take_before` design alternative

§ 4.1.8 Changes since P3705R1

- Remove `unchecked_take_before` and pass-by-value design alternatives
- Add polls and minutes from Sofia 2025 SG9 review
- Move `null_term` to namespace `std::views`
- Add feature test macro
- Add naming discussion
- Use `iter_value_t<I>()` over `iter_value_t<I>{}` to avoid `initializer_list` interaction

§ 4.2 Relevant Polls/Minutes

§ 4.2.1 SG9 review of P3705R1 on 2025-06-18 during Sofia 2025

- [Minutes](#)

POLL: We would like something like the proposed `null_sentinel` regardless of the addition of a hypothetical `unchecked_take_before` view that may come in the future.

Outcome: No objection to unanimous consent

POLL: Always pass the iterator by reference to `operator==`, move `null_term` to namespace `std::views`, add a discussion about alternative names, and forward P3705R1 with those changes to LEWG for inclusion in C++29.

SF	F	N	A	SA
9	7	0	0	0

Of Authors: 1

Author's Position: SF

Attendance: 18 (2 abstentions)

Outcome: Unanimous consent

§ 4.2.2 SG9 review of D2728R4 on 2023-06-12 during Varna 2023

— [Minutes](#)

POLL: Move `null_sentinel_t` to `std::` namespace

SF	F	N	A	SA
1	3	1	0	0

Of Authors: 1

Author's Position: F

Attendance: 9 (4 abstentions)

Outcome: Consensus in favor

POLL: Remove `null_sentinel_t::base` member function from the proposal

SF	F	N	A	SA
0	4	1	0	0

Of Authors: 1

Author's Position: F

Attendance: 8 (3 abstentions)

Outcome: Consensus in favor

§ 4.2.3 SG16 review of P2728R3 on 2023-05-10 (Telecon)

— [Minutes](#)

POLL: Separate `std::null_sentinel_t` from P2728 into a separate paper for SG9 and LEWG; SG16 does not need to see it again.

SF	F	N	A	SA
1	1	4	2	1

Attendance: 12 (3 abstentions)

Outcome: No consensus; author’s discretion for how to continue.

§ 5 References

[P2210R2] Barry Revzin. 2021-03-05. Superior String Splitting.

<https://wg21.link/p2210r2>

[P2728R0] Zach Laine. 2022-12-24. Unicode in the Library, Part 1: UTF Transcoding.

<https://wg21.link/p2728r0>